Software Architecture and Design Specification

Snake Game

Project: Snake Game (Console and Graphics Interface)
Version: 1.0
Author: Pradyumna
SRN: PES2UG24CS359
Course: Software Engineering – Mini Project
Date: 03-10-2026

Project: Snake Game (Console and Graphics Interface)
Version: 1.0
Author: Noel
SRN: PES2UG24CS330
Course: Software Engineering – Mini Project
Date: 03-10-2026

Project: Snake Game (Console and Graphics Interface)
Version: 1.0
Author: Sudeshna
SRN: PES2UG24CS313
Course: Software Engineering – Mini Project
Date: 03-10-2026

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Snake Game mini project.

1.2 Scope

Covers the Snake Game's core player-facing functionality: interface mode selection (console or graphics), snake movement and control, food generation and growth, obstacle/wall/self-collision detection, score tracking, and session control (pause, resume, restart, exit).

1.3 Audience

Developers, QA engineers, course instructor/evaluator, and future maintainers of the codebase.

1.4 Definitions

FR, NFR, SRS, RTM, CLI, GUI, UML, ADR (Architecture Decision Record).

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including the UML component diagram, UML sequence diagrams, interface definitions, and a lightweight threat model, consistent with the project's SRS and Test Plan.

2.2 Related Documents

Snake Game SRS v1.0, Snake Game Software Test Plan v1.0, Snake Game Architecture Selection Justification v1.0.

3. Architecture

3.1 Goals & Constraints

Goals: smooth real-time gameplay (tick processing ≤ 50 ms), portability across Windows and Linux, and maintainable separation between rendering and game logic. Constraints: must be implemented in C/C++; limited to libraries available on lab machines (conio.h/ncurses for console, graphics.h/SDL2 for graphics); offline, single-player, in-memory only — no network or persistent database.

3.2 Stakeholders & Concerns

- Player: responsive controls, clear visual/text feedback, fair and consistent difficulty
- Course Instructor/Evaluator: traceability from requirements to design to tests
- Development Team (4 members): modularity, so each member can work on a module independently
- Future Maintainers: ability to extend the game (e.g., new interface or feature) without rewriting core logic

3.3 Component (UML) Diagram

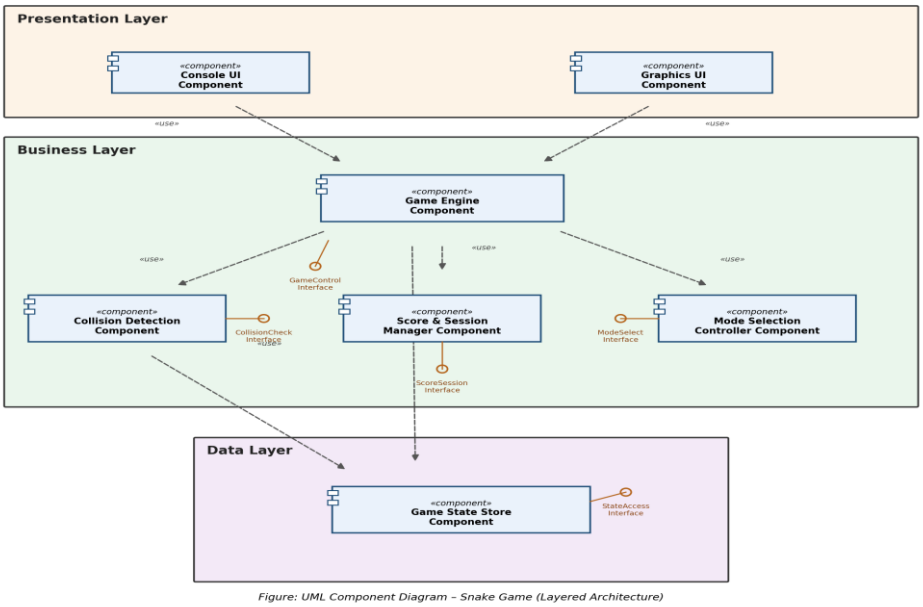


Figure 3.3: UML Component Diagram – Snake Game (Layered Architecture)

3.4 Component Descriptions

- Console UI Component: Renders the game board as a text/ASCII grid in a terminal and captures raw keyboard input for console mode.
- Graphics UI Component: Renders the game board as a graphical window using shapes/sprites and captures keyboard events for graphics mode.
- Mode Selection Controller Component: Presents the startup menu and initializes whichever interface (console or graphics) the player selects.
- Game Engine Component: Owns the core game loop — movement and food/growth logic — and coordinates calls to Collision Detection, Score & Session Manager, and Game State Store on every tick.
- Collision Detection Component: Checks the snake's head position against walls, obstacles, food, and its own body, returning the result to the Game Engine.
- Score & Session Manager Component: Tracks the session score, handles game-over/session-end logic, and manages pause/resume/restart state.
- Game State Store Component: Holds the in-memory state of the current session — snake body segments, food position, obstacle positions, and current score.

3.5 Chosen Architecture Pattern and Rationale

Layered Architecture was chosen for clear separation between presentation, game logic, and session state. It allows two different front-ends (console and graphics) to share a single Game Engine through one interface, keeps the performance-critical game loop decoupled from rendering cost, and centralizes state mutation for easier bounds-checking and security review. Microservices was rejected as unnecessary operational overhead for an offline, single-process, single-player application, and Client-Server was rejected because there is no server — the entire game runs locally for one player.

3.6 Technology Stack & Data Stores

Language: C/C++. Console rendering: conio.h (Windows) or ncurses (Linux). Graphics rendering: graphics.h/WinBGIm or SDL2. Compiler: GCC/G++ or MSVC. Data store: none persistent — all session state (snake, food, obstacles, score) is held in memory for the duration of a single play session and discarded on exit.

3.7 Risks & Mitigations

Risk	Mitigation
Inconsistent gameplay speed across machines of different performance	Fixed game-loop tick rate decoupled from rendering, enforced in the Game Engine
Behavioural differences between console and graphics builds	Both UIs call the same Game Engine through a single shared interface; test both modes on both target OS platforms
Memory-safety defects (buffer overflows/out-of-bounds access) in C/C++	Centralize state mutation in the Business Layer with bounds-checked array access (see 3.9)

3.8 Traceability to Requirements

Requirement ID	Component
SNK-F-001 (Interface mode selection)	Mode Selection Controller
SNK-F-004 / SNK-F-006 (Movement & direction control)	Game Engine
SNK-F-009 (Food consumption & growth)	Game Engine + Collision Detection
SNK-F-012 / SNK-F-013 (Wall/obstacle collision)	Collision Detection
SNK-F-014 (Self-collision)	Collision Detection
SNK-F-015 / SNK-F-016 (Score tracking & display)	Score & Session Manager
SNK-F-018 (Restart session)	Score & Session Manager + Game State Store
SNK-NF-001 (Tick processing ≤ 50 ms)	Game Engine
SNK-SR-001 / SNK-SR-002 (Bounds-checking, safe input)	Collision Detection + Console/Graphics UI

3.9 Security Architecture

Threat Modeling (STRIDE), adapted for an offline single-player local application:

- Spoofing: Not applicable — single local player, no network identity to spoof.
- Tampering: Primary risk is memory corruption via buffer overflow or out-of-bounds array writes in C/C++; mitigated by bounds-checked array access in the Collision Detection and Game Engine components (SNK-SR-001).
- Information Disclosure: Minimal risk — the application holds only in-memory game state, with no sensitive data, persistent file, or network output.
- Denial of Service: Malformed, rapid, or invalid keyboard input could hang or crash the application; mitigated by input validation, discarding unmapped keys (SNK-SR-003), and avoiding unsafe input functions such as gets() (SNK-SR-002).
- Elevation of Privilege: Not applicable — the application runs entirely within the player's own user-level process, with no elevated privileges or multi-user access control.

4. Design

4.1 Design Overview

The Snake Game is designed with layered modules for separation of concerns and easier maintenance: a Presentation Layer that only renders state and forwards player input, a Business Layer that owns all game rules and state mutation, and a Data Layer that holds the current session's in-memory state.

4.2 UML Sequence Diagrams

Two sequence diagrams covering the project's two core flows: everyday gameplay (movement and eating food), and the collision/game-over/restart flow.

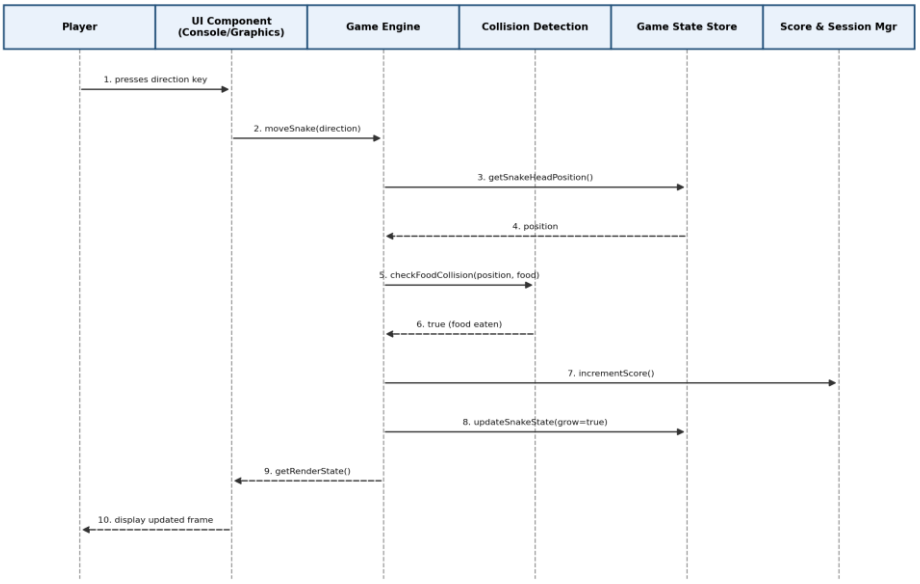


Figure 4.2a: Sequence Diagram – Move Snake & Eat Food

Figure 4.2a: Sequence Diagram – Move Snake & Eat Food

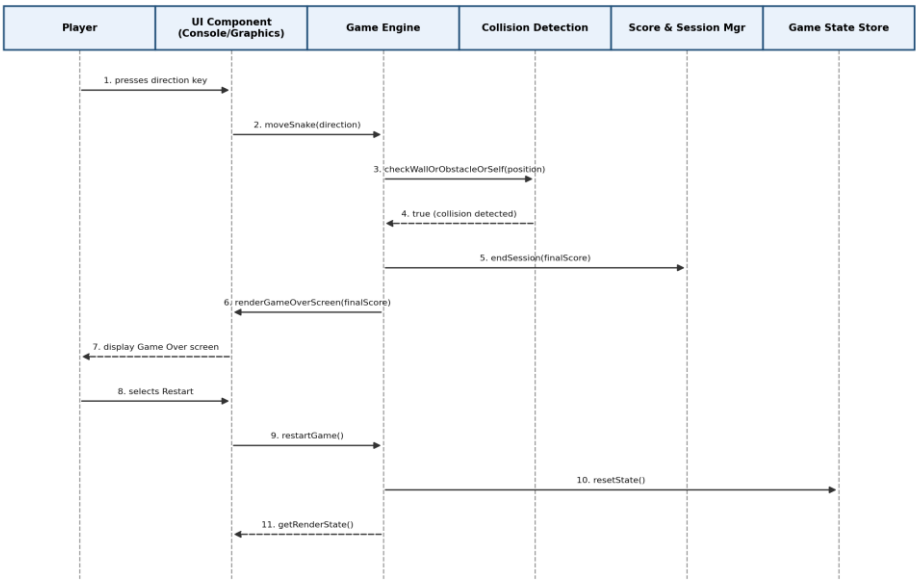


Figure 4.2b: Sequence Diagram – Collision, Game Over & Restart

Figure 4.2b: Sequence Diagram – Collision, Game Over & Restart

4.3 API Design

As a local, offline C/C++ application with no network layer, “API” here refers to the in-process function interfaces between components (as shown in the Component Diagram), rather than HTTP endpoints. Interface definitions for the two most heavily used components:

Interface: GameControlInterface (provided by Game Engine Component)

- void startGame(InterfaceMode mode) — Request: selected mode (console/graphics). Response: initializes a new session. Errors: invalid mode value is rejected and the startup menu is re-shown.
- void moveSnake(Direction dir) — Request: direction enum. Response: updates snake direction for the next tick. Errors: a direction reversing directly into the snake's own body is ignored (no-op).
- void pauseGame() / void resumeGame() — Request: none. Response: toggles session paused state. Errors: calling resumeGame() when not paused is a no-op.
- void restartGame() — Request: none. Response: resets snake, score, food, and obstacles to initial state. Errors: none expected.
- GameState getRenderState() — Request: none. Response: snapshot of snake positions, food position, obstacle positions, score, and game-over flag for rendering. Errors: none expected.

Interface: CollisionCheckInterface (provided by Collision Detection Component)

- bool checkWallCollision(Position head) — Request: head coordinate. Response: true/false. Errors: out-of-range coordinates are rejected before lookup and treated as a collision (fail-safe).
- bool checkObstacleCollision(Position head) — Request: head coordinate. Response: true/false. Errors: same bounds-check as above.
- bool checkSelfCollision(Position head, SnakeBody body) — Request: head coordinate and current body segments. Response: true/false. Errors: none expected.
- bool checkFoodCollision(Position head, Position food) — Request: head coordinate and food coordinate. Response: true/false. Errors: none expected.

4.4 Error Handling, Logging & Monitoring

Invalid or unmapped keyboard input is discarded without altering game state or crashing the application (SNK-SR-003). No sensitive information is logged, since the application holds no personal or sensitive data. An optional debug log may record per-tick processing time to monitor against the ≤ 50 ms performance budget (SNK-NF-001) during development and testing.

4.5 UX Design

Graphics mode: clear, high-contrast shapes for the snake, food, and obstacles, with the current score always visible on screen and a simple start/game-over menu. Console mode: a fixed-size ASCII grid with distinct characters for the snake, food, and obstacles, a visible on-screen key legend (arrow keys/WASD to move, P to pause, Q to quit), and a clearly separated score line.

4.6 Open Issues & Next Steps

- Persistent (cross-session) high-score storage — out of scope for v1.0, noted as a possible future enhancement
- Difficulty progression (e.g., increasing speed or obstacle count over time) — not included in the current scope
- Potential future support for additional interface modes without changing the Game Engine, given the layered design

5. Appendices

5.1 Glossary

FR, NFR, SRS, RTM, CLI, GUI, UML, STP, Tick (one game-loop update cycle).

5.2 References

Snake Game SRS v1.0; Snake Game Software Test Plan v1.0; IEEE 42010 (Systems and software engineering — Architecture description).

5.3 Tools

Custom Python-based diagram generation (component and sequence diagrams); draw.io (optional, for further editing); Git for version control.